

RECALL SEARCH

Personal Knowledge Search | Project Brief | Java + Lucene
For: You & Your Project Partner | Confidential

The Vision

Recall Search is a **desktop application** that turns your entire computer into a searchable knowledge base. Think Google — but for *your own files, PDFs, code, screenshots, and notes*. Type a word, get results in milliseconds. No boot lag. No heavy RAM. Works on old laptops.

Feasibility Verdict

Requirement	Verdict	Note
Ultra-fast search	■ YES	Apache Lucene — faster than Windows Search
RAM < 100 MB	■ YES	Lucene uses ~20-40 MB; avoid Spring Boot overhead
No boot-time delay	■ YES	System-tray app, always running, instant popup
Read inside PDFs/Docs	■ YES	Apache Tika — 1000+ formats supported
OCR (images/screenshots)	■■ YES*	Tess4J works but runs in background, not real-time
Lightweight AI	■■ HARD	Add in Phase 2; local models need 2-4 GB RAM spike
Windows + Linux	■ YES	Pure Java runs everywhere; tray icons need minor tweaks
Code search	■ YES	Lucene indexes .java/.py/.xml — huge dev feature
< 1 GB disk	■ YES	Lucene index = ~25% of text size; very compact
Grammarly-style tray icon	■ YES	Java SystemTray API + global shortcut

Tech Stack

Technology	Role	What it does (2-line)
Apache Lucene	Search Engine	The core brain. Indexes text from any file and returns search results in milliseconds. Used by Elasticsearch internally — battle-tested, tiny footprint (~5 MB jar).
Apache Tika	Content Extractor	Reads inside 1000+ file types: PDF, Word, Excel, PowerPoint, HTML, ZIP. Extracts pure text so Lucene can index it — one library replaces ten parsers.
Tess4J (Tesseract)	OCR Engine	Converts images & screenshots to searchable text using AI-trained models. Run as background job so it never freezes the UI; accuracy is ~95% on clean images.
Java WatchService	File Watcher	Part of Java itself (no extra library). Detects new/changed/deleted files instantly. Triggered indexing — only re-indexes what changed, never the whole drive.
SQLite	Metadata DB	Ultra-lightweight database (single file, zero config). Stores file paths, metadata, tags. Perfect for desktop apps — used by WhatsApp, Firefox, and Android itself.
JavaFX / Swing	Desktop GUI	JavaFX: modern, styled, smooth animations. Swing: simpler, lighter, easier to learn. Both are pure Java — no web tech needed. Recommendation: start with Swing.
Java SystemTray	Tray Icon	Built into Java — places app icon in system taskbar like Grammarly or Dropbox. Global shortcut (e.g. Ctrl+Space) opens search box from anywhere on screen.

Vosk (optional)	Audio/Voice Search	Offline speech-to-text — ~50 MB model, no internet required. Phase 3 feature: search inside lecture recordings by spoken words.
Ollama (Phase 2)	Local AI	Runs small LLMs (Gemma 2B, Phi-3 Mini) completely offline on your laptop. Allows natural language queries: 'find JWT project from last week'.

Build Phases (Start Here → Grow Slowly)

Phase	Feature	What You Build	Difficulty
Phase 1 2-4 weeks	File Name Search	Search by filename, extension, date, size using Lucene. Command-line first. No GUI yet. Master Lucene basics.	■■■■■
Phase 2 3-5 weeks	Read Inside Files	Add Tika — now Lucene indexes PDF & Word content. Simple Swing GUI: search box + results list.	■■■■■
Phase 3 2-3 weeks	Background Indexer	WatchService monitors folders. Auto-index new files. Progress bar, system tray icon, global shortcut.	■■■■■
Phase 4 3-4 weeks	OCR Support	Tess4J indexes screenshots & image text in background. Search 'JWT error' and find that screenshot instantly.	■■■■■
Phase 5 Future	AI Natural Language	Ollama + local LLM for: 'find Spring project from last week'. Vector embeddings for semantic (meaning-based) search.	■■■■■

Architecture Overview

NO Spring Boot needed for Phase 1-3. Spring Boot is a web server framework — using it for a desktop file search adds 300+ MB of overhead and slows startup. Instead use pure Java with modular packages.

Package structure:

- `core.indexer` — reads files, extracts text, writes to Lucene index
- `core.searcher` — takes user query, returns ranked results from Lucene
- `core.watcher` — WatchService thread, detects file changes, triggers re-index
- `core.ocr` — Tess4J thread pool, runs image-to-text as background job
- `ui.swing` — search bar, results panel, settings, tray icon
- `db.sqlite` — file metadata, last-indexed timestamps, user tags

Spring Boot enters in Phase 5 only if you expose search as a REST API (for browser extension or mobile). Even then it's optional.

■■ Honest Warnings & Reality Checks

■ Multi-threading is REQUIRED

Background indexing, OCR, and file watching all run on separate threads. If you block the main thread, the GUI freezes. You MUST learn Java ExecutorService and thread safety. This is significantly harder than CRUD.

■ Lucene has a learning curve

It's not plug-and-play like Spring Data JPA. You write index writers, analyzers, query parsers manually. Budget 1-2 weeks just to understand Lucene basics before writing real code.

■ Spring Boot — Don't use it yet

Every Spring Boot app starts a full web server. On old laptops this adds 2-5 second startup, 150+ MB RAM, and complexity you don't need. Pure Java is the right choice for Phases 1-4.

■ AI model = RAM spike

Ollama + Gemma 2B needs ~2 GB RAM minimum. On an 8 GB laptop that leaves only 6 GB for your OS and other apps. This is Phase 5 only — do NOT add it early.

■ **OCR is slow by design**

Tesseract takes 1-3 seconds per image page. Index 500 screenshots = 10-25 minutes. Always run it as a background thread with a queue. Never in the UI thread.

■ **Cross-platform gotchas**

Tray icon looks different on Windows vs Linux. Global shortcuts may conflict with OS shortcuts. Test on both early — don't leave it for last.

■ **Corrupted files will crash your indexer**

Real-world files include broken ZIPs, password-protected PDFs, and 10 GB video files. Wrap every file-read in try-catch with timeout limits, or one bad file will kill the indexing thread.

How Hard vs CRUD?

Skill	CRUD App	Recall Search
Java	Basic OOP, loops	Threads, concurrency, ExecutorService
Data Storage	MySQL + JPA (easy)	Lucene index + SQLite (learn new APIs)
Background work	None needed	CRITICAL — all indexing is background
Error handling	Simple try-catch	Corrupt files, timeouts, stream errors
Testing	JUnit on REST endpoints	Unit test indexing logic & thread safety
Memory mgmt	Spring handles it	YOU control buffers, caches, heap size
Overall	■■■■■	■■■■■ (Phase 1-3) / ■■■■■ (Phase 5)

Minimum Viable Product (Build This First)

Your MVP goal: **A working search tool that indexes user-selected folders and searches inside PDF/Word files, with a simple Swing GUI and system tray icon.**

- File name search
 - Search inside PDF, Word, TXT
 - Background indexing (WatchService)
 - Simple Swing GUI — search box + results
 - System tray icon + Ctrl+Space shortcut
- NOT in MVP: OCR, Audio, AI, Plugins, Video search

Memory & Performance Targets

Component	RAM Usage	Disk Usage
JVM (Java runtime)	~40 MB	—
Lucene index (50K files)	~15-25 MB	~500 MB index
SQLite metadata DB	~5 MB	~50 MB
Swing GUI	~10 MB	—
Tika parser (while indexing)	~30 MB peak (then freed)	—
Tesseract OCR (bg thread)	~50 MB peak (then freed)	~30 MB models
TOTAL (Phase 1-3)	~70-90 MB ■	~600 MB ■
WITH AI (Phase 5)	~2.5 GB ■■	~2.5 GB ■■

Your First 3 Steps (This Week)

Step 1 — Learn Lucene basics

Watch: 'Apache Lucene Tutorial' on YouTube (search it). Goal: write a Java program that indexes a folder and searches filenames. 3-4 days.

Step 2 — Add Tika for PDF/Word reading

Add Tika dependency in Maven/Gradle. Feed it a PDF, print extracted text. Then feed that text to Lucene. 2-3 days.

Step 3 — Simple Swing GUI

JFrame + JTextField (search box) + JList (results). Wire it to your Lucene searcher. Ship this as your MVP. 1 week.